LeetCode Practice

1 Easy: Backtracking

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

1
EASY

Patterns (in order)

1. Backtracking (1)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

1 questions | tap a title to jump to it

Easy questions (1) Go >

Backtracking (1) Go >

E 1. Binary Tree Paths LC 257

Easy questions (1)

EASY | TOPIC 1 OF 1

Backtracking

1 Easy question start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Binary Tree Paths LC 257

EASY | Backtracking | LeetCode #257 | 1 of 1 | 1/1

Binary Tree Paths

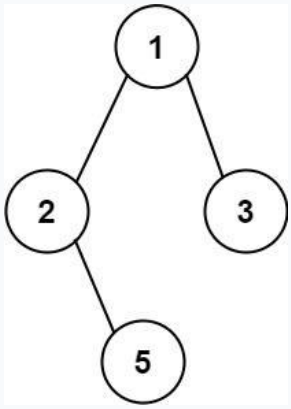
QUESTION

Given the root of a binary tree, return all root-to-leaf paths in any order.

A leaf is a node with no children.

Example 1

Input	`root = [1,2,3,null,5]`
Output	`["1->2->5","1->3"]`



Example 2

Input	`root = [1]`
Output	`["1"]`

CONSTRAINTS

- The number of nodes in the tree is in the range [1, 100].
- 100 <= Node.val <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return all root-to-leaf paths in a binary tree as strings like '1->2->5'. Right?"

"A few quick questions:

Any order is fine? Yes. Leaf is a node with no children? Yes."

"Let me walk: root=[1,2,3,null,5] -> ['1->2->5','1->3'] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def paths(node, prefix):
    extend the prefix string with the node value
    at a leaf record it
    otherwise recurse into both children with the
        longer string
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n * h)	O(n * h)

NOTE

Not slow in the big picture (the output is a list of paths). Concatenating builds a new string at every node; passing a shared list and joining only at leaves avoids the copies.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DFS collecting root-to-leaf paths as strings — O(n). Walk-through:"

`def dfs(node, path):`

If node is None: return.

`path += str(node.val).`

If node.left is None and node.right is None: <- leaf

`result.append(path).`

Else:

`dfs(node.left, path+'->').`

`dfs(node.right, path+'->').`

`dfs(root, '').`

WHY append only at leaf: a path is root-to-leaf, not root-to-any-node.

WHY pass path by value (string): each recursive call gets its own copy — no backtracking needed.

(Alternatively use a list and backtrack, but string concatenation is simpler here.)

Time: O(n * h) for string ops. Space: O(h) recursion depth.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _257_BinaryTreePaths:
    def binaryTreePaths(self, root):
        result = []
        def dfs(node, path):
            path.append(str(node.val))
            if not node.left and not node.right:
                result.append('->'.join(path))
            else:
                if node.left: dfs(node.left, path)
                if node.right: dfs(node.right, path)
            path.pop()
        if root:
            dfs(root, [])
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace binaryTreePaths on [1,2,3,null,5]."

`dfs(1, '1'): not leaf (has children 2,3).`

`dfs(2, '1->2'): has right child 5.`

`dfs(5, '1->2->5'): leaf -> result.append('1->2->5').`

PHASE 5 TEST & DEBUG

`dfs(3, '1->3'): leaf -> result.append('1->3').`

Result: ['1->2->5','1->3'] ok

WHY string concatenation (no backtrack needed): strings are immutable in Python — each

recursive call gets its own copy of the path string, so no state to undo.

Edge: single root [1] -> dfs(1,'1'): leaf -> ['1'] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n*h)	**O(h)**
n nodes, each path up to h long	call depth

Follow-up: Path Sum II (#113) — same DFS but filter by target sum.

Follow-up: Smallest String Starting From Leaf (#988) — compare paths lexicographically.

IN YOUR APP | Expose | Editorial | Community